

tracey: Components, Interactions, and the Choices Behind Them

Shubhro, Radha, Gangasagar and Arav
Software Design Practical, Assignment 2

1. What we are designing

The whole system is built around one question. Say a student is in their second year and wants to add a mathematics minor. Can they still do it from where they are? And if they can't, which course is the one that ruled it out?

We worked that out in the last assignment. Seven term timetables gave us 474 courses, and 181 of them have only been offered once. The courses a major requires sit in prerequisite chains, and those chains run in fixed trimester slots. Put those together and there is a cut-off for declaring a subject that isn't written down anywhere. This document is about how we are actually building the thing, and why each part sits where it does. Where we looked at another option and didn't take it, we have said what it was.

The work splits into three jobs. They are quite different from each other, and that is mostly what decides how the system is laid out.

- Turning spreadsheets and web pages into a catalogue. This runs a few times a year, nobody is waiting for it, and it can take minutes.
- Answering questions about that catalogue. It only reads, and the answer is the same for every student, so we can work it out once and reuse it.
- Holding one student's plan and checking it again every time they change it. This is per-student, there are a lot of writes around registration, and it has to feel instant.

2. How a web application is put together

It helps to set out the general picture first, because the same trade-off comes up in every section after this.

A web application runs on three machines that act like one.

The browser runs on the student's own laptop. Nothing has to go over the network, so anything that happens here is instant, which makes it the right place for work that needs to feel immediate. It has three limits. We can't trust it, because anyone can open the developer tools and change what it sends. It can't hold anything that is shared between people. And anything it saves locally is gone if the student clears their browser or uses a different device.

The server runs somewhere we control. That makes it the only place we can trust, since nobody else can change the code, and the only place that sees everyone's data. It is also the slow one. Every request costs a round trip of 50 to 300 milliseconds, which is much too slow to happen in the middle of a drag.

The database sits behind the server. We keep them separate on purpose. The server should be throwaway, so we can restart it or redeploy it without losing anything, and the database is the part that has to survive that. It is also the only part that can enforce rules about the data itself, so that two requests arriving at the same time can't leave the records in a state neither of them wanted.

There is a fourth piece outside all of this. **Batch work** is anything slow that runs on a schedule

with nobody waiting for it. Scraping the university's pages goes here, because a student shouldn't have to wait for krea.edu.in to be up.

So the same question comes up again and again. Can this happen locally, with data the browser already has? Or does it need the server, and can it afford the wait? For tracey the answers are mostly obvious.

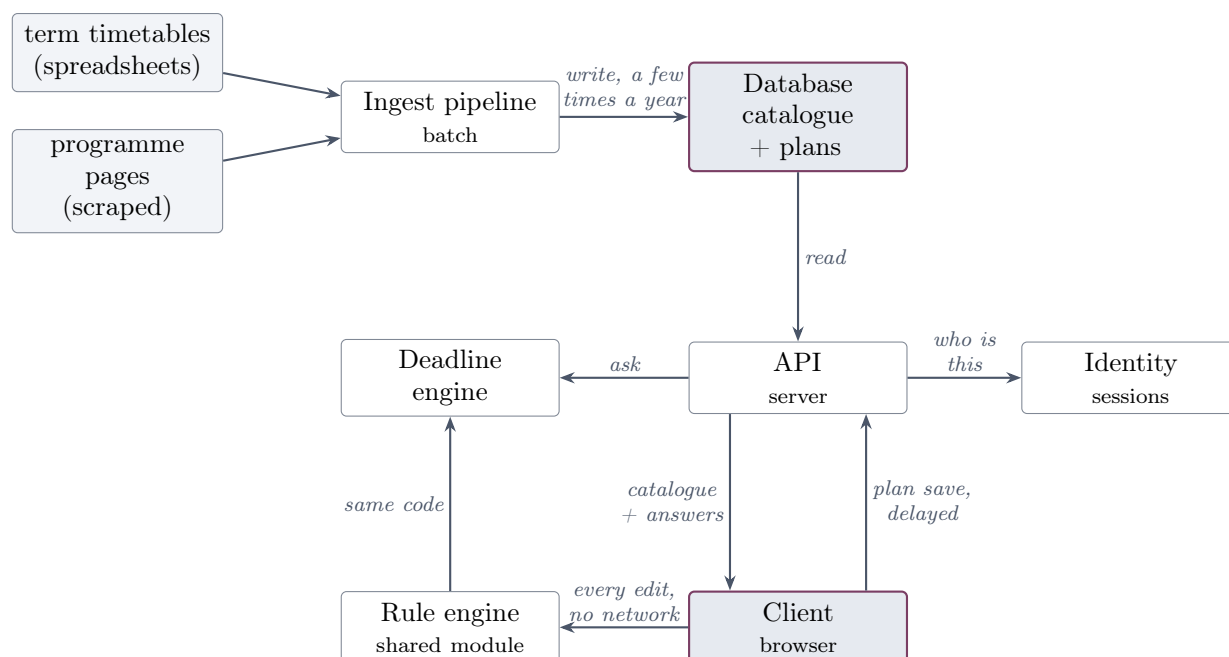
Work	Why it goes there	Where
Scraping and parsing	Slow, scheduled, nobody waiting	Batch
Storing the catalogue	Must survive restarts, read by everyone	Database
Computing a deadline	Needs the whole catalogue, same for everyone	Server
Re-checking a plan on a drag	Must beat a 16 ms frame, data already local	Browser
Saving a plan	Must survive a cleared browser	Server, then database
Deciding who you are	Cannot be trusted to the browser	Server

3. The components

Component	Responsibility	Where it runs
Ingest pipeline	Read the term timetables and scrape the thirteen programme pages. Resolve prerequisite prose to course codes and flag whatever it cannot resolve for a human. Write the catalogue.	Batch, a few times a year
Catalogue store	The courses, their offerings by trimester, the prerequisite graph, the requirement sets, the credit tables. One version per pipeline run.	Database
Deadline engine	Given a programme and a starting trimester, compute the earliest each required course can be taken, the earliest the programme can finish, and the last trimester it can still be declared.	Server
Rule engine	The rules a plan must satisfy: credit floor and cap, prerequisite order, offering trimester, year eligibility, foundation core placement, audit limits, substitutes, credit totals. Returns violations in plain words.	Shared module, mostly in the browser
Identity	Sign-in, sessions, and the read-only share links that let a mentor see a plan without an account.	Server
Plan store	One student, several named plans, each a set of (course, year, trimester) placements plus grades and audit marks.	Database
API	The contract between browser and server. Catalogue read, deadline query, plan create/read/update/delete, share link, sign-in callback, sign-out.	Server
Client	The screen the student actually uses: subject pickers, a course catalogue, a year $\times$ trimester grid they drag courses around, and a panel that explains what is currently wrong with the plan.	Browser

One part is shared instead of belonging to one side. The rule engine is a single module that both the browser and the server use. That way the server can't call a plan illegal after the browser has already said it is fine. It is a bit awkward, since it has to be written in a language both sides can run, but two separate copies would drift apart within a term and the tool would start disagreeing with itself.

4. How the components talk



Four interactions matter, and they work quite differently from each other.

A student drags a course into a different trimester. The board tells the plan that the course has moved, the plan makes the change, the rule engine checks every rule over the whole plan, and the panel redraws. None of that touches the network, which is the whole reason the rule engine runs in the browser. A save is queued and sent about a second later, and if it fails the plan is still right on screen and still saved locally. The student never waits on a server to find out their trimester is overloaded.

A student asks whether a minor is still possible. The picker sends the programme and the student's current trimester to the server. The deadline engine runs it against the catalogue and sends back two things: the last trimester the programme could still have been declared in, and the first course that fails. The screen writes that out as a sentence. This is the one question the browser can't answer by itself, because it needs the whole catalogue and the whole prerequisite graph, not just what is on screen.

A student signs in. Usually they have been planning for ten minutes already, with the plan sitting in the browser. Signing in hands off to the identity provider, which sends them back to a callback route on the server, and the server then finds or creates the student record and starts a session. The client posts the plan it was holding, and the server saves it to the account as a named plan. Nothing is thrown away or overwritten. The other option would mean losing work for signing in late, which we don't want.

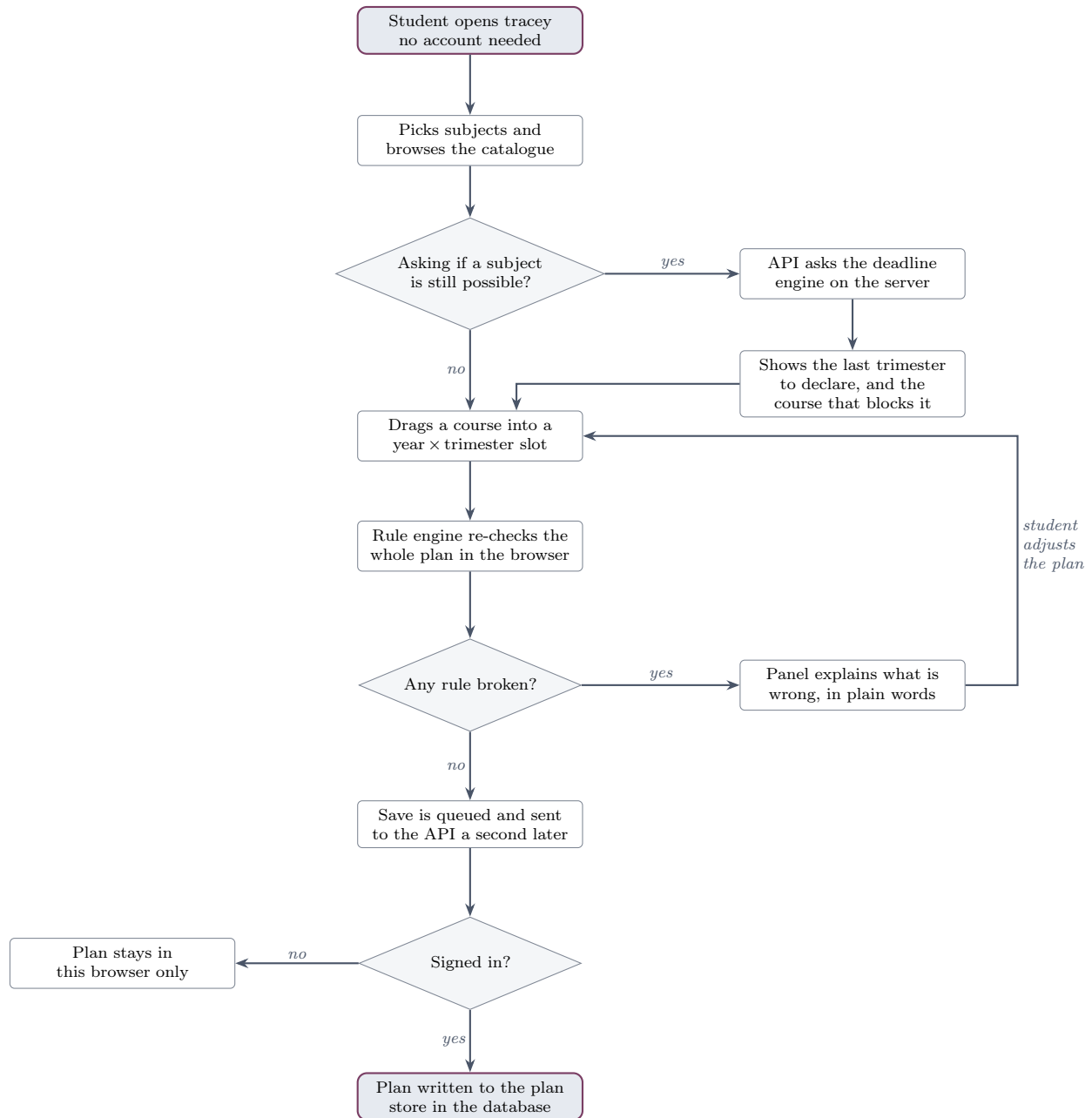
A new timetable comes out. The pipeline runs and writes a new catalogue version in one transaction, and then compares it against the old one. A course that moved from T2 to T1 changes a chain, a changed chain changes a deadline, and a changed deadline can quietly break plans students have already saved. So the last thing the pipeline does is run the rule engine over every stored plan and mark the ones that no longer work. Those students get a notice the next time they open the app. Without that, the app would keep showing last term's answer and nobody would know.

4.1 The two paths through the system

Two kinds of people use tracey. Students edit plans in the browser, and whoever looks after the catalogue runs the pipeline a few times a year. The two flowcharts below follow each of them

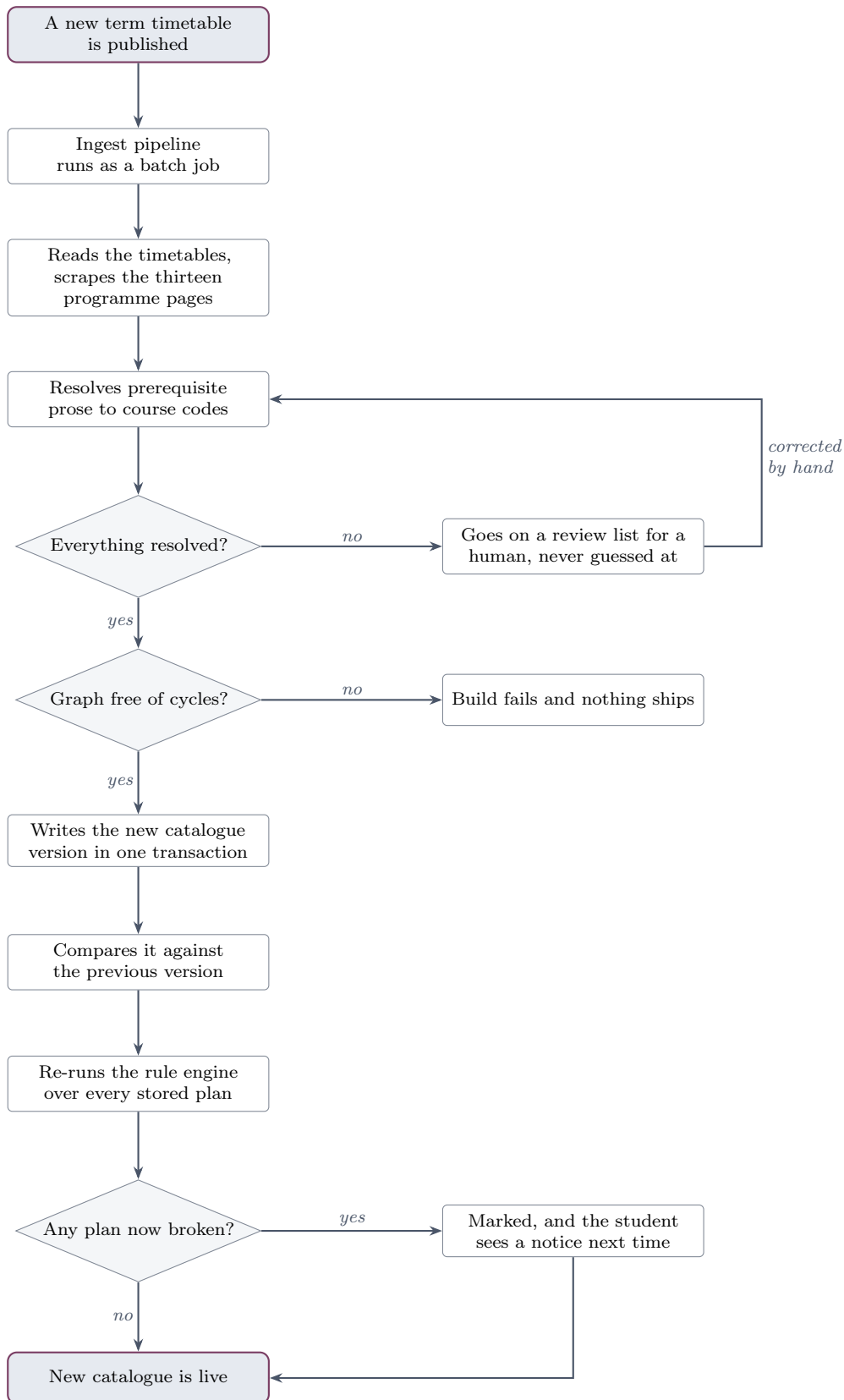
through the components from section 3. Neither one uses anything we haven't already described.

Student flow



The loop in the middle is the main thing to notice. Dragging a card, checking the rules and showing the warning all happen inside the browser, so that loop never touches the network. Only the two branches off to the side, the deadline question and the save, leave the laptop.

Catalogue maintainer (admin) flow



The two places this can fail are on the right, and neither of them guesses. A prerequisite the pipeline can't resolve waits for a person, and a cycle in the graph stops the build. The last three steps before the end are there to protect plans students have already saved.

5. The database

Why use a database. A program’s variables are gone when the program stops, so anything that has to last longer than one request has to be written somewhere. The simplest option is files, and files are fine until three things start to matter. Two users write at the same time and overwrite each other. Questions start covering more than one record, like “which courses does this requirement name, and when did each one run”. A write stops halfway and leaves a mess. A database handles all three. The main families are built for different things: relational databases store rows in tables and join across them, document databases store nested objects and try to avoid joins, and graph databases store nodes and edges and are built for following connections.

The choice: a relational database, PostgreSQL.

Our data is relational and most of our questions are joins. A requirement points at courses. A course has several codes, because of cross-listing. It has many offerings. It has prerequisite groups, and the members of those groups are courses too. “What does a mathematics minor require, and when has each of those run” is about four joins, which is what SQL is for. Two Postgres features are useful here. Recursive queries get a course’s full set of prerequisites in one round trip instead of one per level. And a JSON column lets us keep the raw scraped page and the raw spreadsheet row next to the parsed version, so if a number looks wrong we can check where it came from without running the scraper again.

```
subject(id, name, discipline, url)
course(id, primary_code, title, credits, subject_id, raw json)
course_code(course_id, code, subject_id)
offering(course_id, academic_year, trimester)
eligibility(course_id, year)
substitute(course_a, course_b)
prereq_group(id, course_id, kind)          -- kind: all_of | any_of
prereq_member(group_id, course_id)
programme(id, subject_id, shape, length) -- shape: single | double
                                         --          | minor | concentration
requirement(id, programme_id, kind, choose_n)
requirement_course(requirement_id, course_id)
student(id, provider_sub unique, email, display_name,
        entry_year, degree_length, created_at)
session(id, student_id, expires_at, created_at, user_agent)
plan(id, student_id, name, updated_at)
plan_item(plan_id, course_id, year, trimester, is_audit, grade)
declaration(plan_id, programme_id)
share_link(token, plan_id, include_grades, revoked_at, created_at)
```

Three of these tables are worth explaining. `course_code` is there because one course can have more than one code, and if we treated those as separate courses a student could end up counting the same credits twice. `offering` has one row for each time a course actually ran, so it is what every deadline we report is based on. Prerequisites need two tables rather than one column, because “MATH202 or MATH230” and “MATH202 and MATH230” are different graphs, and putting them in one column would lose the difference the deadline calculation depends on.

We will also precompute a view holding each course’s set of observed trimesters, refreshed at the end of every pipeline run. The deadline engine reads it constantly, and working it out on every query would turn a lookup into a scan.

What we didn’t pick. SQLite is the obvious cheap option. A catalogue of 474 courses fits in one file, so for reads it would be fine. Writes are the problem: hundreds of browsers will be saving plans in the same fortnight around registration. We would also rather run migrations against a server

than pass a new file around. A document database has the opposite problem. Without joins we would copy each course into thirteen programme documents, and those copies would disagree after the first correction, which is the sort of mismatch tracey is supposed to get rid of. A graph database was the harder one to turn down. Prerequisites really are a graph, and the chain query would be one line. But the graph is small, around 474 nodes and 600 edges, which we can walk in memory in microseconds, and everything else is plain tables: credit tables, offerings, plans, grades. One ordinary database is simpler than a specialised one plus a second one for everything it can't do.

6. Accounts and sign-in

Authentication and authorisation. Two different things tend to get called by the same name. Authentication is working out who someone is. Authorisation is deciding what they are allowed to touch. They need separate answers, and getting the second one wrong is what does the real damage.

For authentication, an application either holds passwords itself or hands the job to someone else. Holding passwords means storing hashes, building a reset flow, and being the party at fault when the database leaks. Delegating means sending the user off to a provider they already have an account with and then trusting the answer that comes back. For keeping someone signed in afterwards there are two options: an opaque session identifier in a cookie, checked against the server's records, or a self-contained signed token, which the server can verify without looking anything up.

The choice: delegate to the university's provider, and keep sessions on the server.

Every SIAS student already has a university Google account, so tracey will use that. It saves the student picking a password, saves us storing a hash, and saves us building a reset flow, and those are three of the likelier things to go wrong in a project this size. Restricting sign-in to the university's domain also handles authorisation for free. If you can sign in, you are a Krea student, and we never have to verify anyone by hand.

We will key the account record on the provider's stable subject identifier rather than on the email address, because addresses get reissued and names change. If we keyed on email, a student could come back to an empty account and lose their plan.

Sessions will be opaque tokens: a random value in a cookie marked httpOnly, Secure and SameSite, with a matching row on the server. A self-contained token would save us that lookup, but the database is already in the path of every plan read, so the lookup costs us nothing extra. What we get in exchange is revocation. Signing out, or losing a laptop, ends the session there and then rather than waiting for it to expire.

Signing in won't be required to start with. A student who opens the link can plan straight away, with the plan held in the browser. We offer sign-in rather than demanding it, and the reasons for doing it are concrete: the plan survives a cleared browser, it follows you to your phone, and you can keep more than one. When they do sign in, whatever plan they were already holding is attached to the account rather than dropped.

Sharing is a separate mechanism. Showing a plan to a mentor shouldn't require the mentor to have an account, and it certainly shouldn't hand them the student's account. So a plan can generate a share link with a random, unguessable token. The link is read-only and can be revoked, and it leaves grades out unless the owner turns them on.

Authorisation then comes down to one rule, short enough to check by reading it. Every plan query is filtered by the session's student. There is exactly one exception, a valid share token, which grants read access to exactly one plan. There will be no administrative view of student plans. Grades are the sensitive part of this data, and they are also the part nobody else needs.

7. The frontend

Two ways to build a website. In the server-rendered style, every interaction asks the server for a new page, and the browser mostly displays what it is given. In the single-page style, the browser loads an application once and after that updates the screen itself, calling the server only for data. The first is simpler and harder to break, and it is better for content. The second is what you need when interaction has to be immediate, because it takes the network out of the loop.

The choice: a single-page application, built with a compiled framework.

tracey is one dense screen that never navigates anywhere. It has subject pickers, a course list, a year $\times$ trimester grid with roughly forty draggable cards on it, and a panel that changes on every drag. One requirement settles the framework question: dropping a card has to re-check the whole plan and repaint before the card lands, which gives us about 16 milliseconds. Asking the server is out on that budget alone. So the rules run locally, and the framework's job is to update the smallest possible piece of the screen.

A compiled framework fits that well. Svelte turns reactive declarations into direct DOM updates at build time, so there is no virtual DOM comparison happening at runtime, and moving one card touches two trimester totals and one list of warnings rather than a tree of components. React would work too, and we would probably prefer it for a bigger application with many screens and several people working on it. Here it costs us a runtime and a fair amount of care about memoisation, to buy back smoothness we would otherwise get for free. A server-rendered approach with something like HTMX is out for a harder reason: it puts a network round trip inside a drag, so the warning would appear a moment after the card lands, which is exactly what makes a tool feel slow. The whole thing will be written in TypeScript. The rule engine is shared between browser and server, and the types are what stop the two copies drifting apart.

8. The algorithms

8.1 Building the prerequisite graph. Prerequisites turn up as English inside a spreadsheet cell. The pipeline normalises the text and pulls out course codes by pattern. Sometimes a title is named instead of a code, and then it matches against the catalogue by normalised token overlap, above a threshold. Anything below the threshold goes onto a review list for a human rather than being guessed at, because a wrong edge produces a wrong deadline that looks perfectly correct. The graph is then checked for cycles with Kahn's algorithm: repeatedly remove the nodes with no remaining prerequisites, and if anything is left over, the data is wrong and the build fails instead of shipping. $O(V + E)$.

8.2 Earliest feasible trimester. This is the core computation, and the whole project rests on it. Number the trimesters from zero, so trimester t is slot $T_{(t \bmod 3)+1}$ of year $\lfloor t/3 \rfloor + 1$. For a course c and a start trimester s :

```
earliest(c, s):
    if (c, s) in memo: return memo[(c, s)]
    t = s
    for each prerequisite p of c:
        t = max(t, earliest(p, s) + 1)      # strictly after the prereq
    while slot(t) not in offered(c) or year(t) not in eligible(c):
        t = t + 1                          # slide to the next real offering
    memo[(c, s)] = t
    return t
```

It is a longest-path computation on a directed acyclic graph, with one addition. The slide step turns it into scheduling with release windows, and the slide is why the numbers come out higher than you

would expect. A chain of five courses isn't five trimesters long if each course only ever runs in one fixed slot. Mathematics is the clearest example. MATH191 runs in T3, MATH230 follows it in T1, then MATH231, then MATH332 in T1 of year three or later, then MATH333 in T3, then MATH434 in T1 of year four. That is six courses spread across four calendar years. With memoisation each node is visited once, so it costs $O(V + E)$ per start value, and the while loop runs at most three times for any course that has ever been offered.

8.3 The declaration deadline. For a requirement set P , $finish(P, s) = \max_{c \in P} earliest(c, s)$. That is monotone in s , since starting later can never let you finish earlier, so the deadline is the largest s with $finish(P, s) \leq H$, where H is 9 or 12 trimesters. Because it is monotone we can binary search over s instead of scanning, which gives $O((V + E) \log H)$. The arg max course comes back with the answer, and that is what lets the interface say “MATH434 is what breaks” rather than just “too late”.

Choice pools need a bit of care. A requirement might say “any three of these five”, and then the contribution isn't the maximum over all five. It is the third smallest of their *earliest* values, because the student will take whichever three finish soonest. Getting that wrong would make subjects like history and sociology look far tighter than they actually are.

8.4 Seeding a plan. Take the required set in topological order and first-fit each course into the earliest trimester at or after its *earliest* value that is legal and still has room under the credit cap. It is greedy, $O(V \log V + VT)$, and not optimal, but it doesn't need to be. It gives the student a legal starting point to rearrange. Where a department publishes its own recommended pathway, as mathematics does, we use that pathway as it is instead.

8.5 Re-checking on every edit. About a dozen rules run over about forty placed courses, and each rule is one pass over the placements, so a full re-check is a few thousand operations and comfortably under a millisecond. That is why we can afford to re-check everything on every drag. The alternative is working out which rules a particular move could have affected, which would be faster, but a case we missed would stop a rule being checked with nothing on screen to say so.

9. Still open

Two things we haven't settled. The first is where the deadline engine runs: on the server, or shipped down to the browser along with the catalogue. That trades about a megabyte of download against a round trip, and we would rather settle it by measuring than by arguing. The second is what the pipeline should do when a course disappears from the timetable altogether. The straightforward thing is to keep the old offering rows, but that assumes the course will run again, and a deadline computed from a course that never comes back is simply wrong.